Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы оптимизации и управления»

ОТЧЕТ
к лабораторной работе №3
на тему:
«ОСНОВНАЯ ФАЗА СИМПЛЕКС-МЕТОДА»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил доцент каф. ИКТ
ГОРБУКОВ Дмитрий Михайлович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Реализовать основную фазу симплекс-метода для задачи линейного программирования в канонической форме. По заданным матрице A, вектору c, начальному базисному допустимому плану x и набору базисных индексов B необходимо либо найти оптимальный план, либо установить неограниченность целевого функционала сверху.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Реализована функция simplex_main_phase, выполняющая симплекс-итерации, и вспомогательная функция update_inverse_matrix, обновляющая обратную базисную матрицу.

Для демонстрации используется пример из методических указаний. Исходная задача линейного программирования:

```
x1 + x2 -> max
-x1 + x2 + x3 = 1
x1 + x4 = 3
x2 + x5 = 2
x1, x2, x3, x4, x5 >= 0
```

Начальный базисный допустимый план:

```
x = [0, 0, 1, 3, 2]
B = [3, 4, 5]
```

На каждой итерации вычисляются вектор потенциалов, оценки delta, входящая переменная, направление z, допустимые шаги theta и новый базис. Полученные результаты имеют следующий вид:

Итерация 1:

```
B = [3, 4, 5]
delta = [1, 1, 0, 0, 0]
j0 = 1
z = [-1, 1, 0]
theta = [inf, 3, inf]
theta0 = 3
k = 2, leaving = x4
x = [3, 0, 4, 0, 2]
```

Итерация 2:

```
B = [3, 1, 5]
delta = [0, 1, 0, -1, 0]
j0 = 2
z = [1, 0, 1]
theta = [4, inf, 2]
theta0 = 2
k = 3, leaving = x5
x = [3, 2, 2, 0, 0]
```

После третьей итерации все компоненты вектора оценок стали неположительными, следовательно текущий план является оптимальным:

```
Оптимальный план x = [3, 2, 2, 0, 0]
Оптимальный базис B = [3, 1, 2]
Значение целевой функции z = 5
Финальный вектор оценок delta = [0, 0, 0, -1, -1]
```

Результат работы программы приведён ниже на рисунке 2.1.

```
--- Итерация 1 ---
Текущий план x = [0. 0. 1. 3. 2.]
Базисные индексы B = [3, 4, 5]
Вектор оценок delta = [1. 1. 0. 0. 0.]
Переменная x1 входит в базис (delta[0] > 0).
Направление z = [-1.  1.  0.]
Минимальный шаг theta0 = 3.0000
Переменная x4 покидает базис (достигается на k=1).
--- Итерация 2 ---
Текущий план x = [3. 0. 4. 0. 2.]
Базисные индексы B = [3, np.int64(1), 5]
Вектор оценок delta = [ 0.  1.  0. -1.  0.]
Переменная x2 входит в базис (delta[1] > 0).
Направление z = [1. 0. 1.]
Минимальный шаг theta0 = 2.0000
Переменная x5 покидает базис (достигается на k=2).
--- Итерация 3 ---
Текущий план x = [3. 2. 2. 0. 0.]
Базисные индексы B = [3, np.int64(1), np.int64(2)]
Вектор оценок delta = [ 0.  0.  0. -1. -1.]
Достигнут оптимальный план.
--- Результат ---
Оптимальный план: [3. 2. 2. 0. 0.]
Оптимальный базис: [3, np.int64(1), np.int64(2)]
Максимальное значение целевой функции: 5.0000
```

Рисунок 2.1 – Результат работы программы

ВЫВОД

В лабораторной работе реализована основная фаза симплекс-метода с эффективным обновлением обратной базисной матрицы. Для тестового примера алгоритм за три итерации пришел к оптимальному плану x = [3, 2, 2, 0, 0] и значению целевой функции z = 5. Полученные значения совпадают с расчетами из методических указаний, что подтверждает корректность реализации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Python Documentation [Электронный ресурс]. – Режим доступа: https://docs.python.org/. – Дата доступа: 09.04.2026.

[2] NumPy Documentation [Электронный ресурс]. – Режим доступа: https://numpy.org/doc/. – Дата доступа: 09.04.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация основной фазы симплекс-метода

```
import numpy as np

def update_inverse_matrix(A_B_inv, new_col, leaving_col_idx):
    """
    Обновляет обратную матрицу A_B^-1, используя метод из lab1.
    Это более эффективно, чем полный пересчет на каждой итерации.

    Args:
        A_B_inv (np.ndarray): Текущая обратная матрица.
        new_col (np.ndarray): Новый столбец, входящий в базис.
        leaving_col_idx (int): Индекс заменяемого столбца в A_B.

    Returns:
        np.ndarray: Новая обратная матрица.
    """
    l = A_B_inv @ new_col

    if np.isclose(l[leaving_col_idx], 0):
        # Эта ситуация не должна возникать в стандартном симплекс-
методе
        # если правила выбора работают корректно.
        raise np.linalg.LinAlgError("Матрица вырождена, невозможно
обновить.")

    l_prime = l.copy()
    l_prime[leaving_col_idx] = -1
    l_hat = (-1 / l[leaving_col_idx]) * l_prime

    E = np.identity(len(l))
    E[:, leaving_col_idx] = l_hat

    return E @ A_B_inv

def simplex_main_phase(A, c, x, B):
    """
    Реализует основную фазу симплекс-метода.

    Args:
        A (np.ndarray): Матрица ограничений (m x n).
        c (np.ndarray): Вектор целевой функции (n).
        x (np.ndarray): Начальный базисный допустимый план (n).
        B (list): Упорядоченный список базисных индексов (m), 0-based.

    Returns:
        tuple: (оптимальный план, базис) или (None, None) если задача
не ограничена.
    """
    m, n = A.shape
    B = list(B) # Копируем, чтобы не изменять оригинал
    x = x.copy()
```

```
    # Рассчитываем A_B_inv для первой итерации
    A_B = A[:, B]
    A_B_inv = np.linalg.inv(A_B)

    iteration = 1
    while True:
        print(f"--- Итерация {iteration} ---")
        print(f"Текущий план x = {np.round(x, 4)}")
        print(f"Базисные индексы B = {[i+1 for i in B]}")

        # Шаги 2, 3: Вычисляем вектор потенциалов u
        c_B = c[B]
        u = c_B @ A_B_inv

        # Шаг 4: Вычисляем вектор оценок delta
        delta = c - u @ A
        print(f"Вектор оценок delta = {np.round(delta, 4)}")

        # Шаг 5: Проверка на оптимальность
        if np.all(delta <= 1e-9): # Сравнение с небольшим допуском
            print("Достигнут оптимальный план.")
            return x, B

        # Шаг 6: Выбор входящей переменной
        j0 = np.where(delta > 1e-9)[0][0]
        print(f"Переменная x{j0+1} входит в базис (delta[{j0}] > 0).")

        # Шаг 7: Вычисляем направление z
        A_j0 = A[:, j0]
        z = A_B_inv @ A_j0
        print(f"Направление z = {np.round(z, 4)}")

        # Шаг 8: Вычисление шага theta
        theta = np.full(m, np.inf)
        for i in range(m):
            if z[i] > 1e-9:
                theta[i] = x[B[i]] / z[i]

        # Шаг 9: Поиск минимального шага theta0
        theta0 = np.min(theta)

        # Шаг 10: Проверка на неограниченность
        if np.isinf(theta0):
            print("Задача не ограничена сверху.")
            return None, None

        # Шаг 11: Выбор исключаемой переменной
        k = np.argmin(theta)
        j_star = B[k]
        print(f"Минимальный шаг theta0 = {theta0:.4f}")
        print(f"Переменная x{j_star+1} покидает базис (достигается на
k={k}).")

        # Шаг 13: Обновление плана x
        x[j0] = theta0
        for i in range(m):
            x[B[i]] -= theta0 * z[i]
```

```
        x[j_star] = 0 # Явно обнуляем, чтобы избежать ошибок
округления

        # Обновление A_B_inv для следующей итерации
        A_B_inv = update_inverse_matrix(A_B_inv, A_j0, k)

        # Шаг 12: Обновление базиса B
        B[k] = j0

        iteration += 1

if __name__ == "__main__":
    # Пример задачи из lab3.pdf (стр. 14)
    c_initial = np.array([1.0, 1.0, 0.0, 0.0, 0.0])
    A_initial = np.array([
        [-1.0, 1.0, 1.0, 0.0, 0.0],
        [1.0, 0.0, 0.0, 1.0, 0.0],
        [0.0, 1.0, 0.0, 0.0, 1.0]
    ])

    # Начальный базисный допустимый план
    x_initial = np.array([0.0, 0.0, 1.0, 3.0, 2.0])
    B_initial = [2, 3, 4] # 0-based индексы (x3, x4, x5)

    print("--- Начало основной фазы симплекс-метода ---")

    # Запуск алгоритма
    optimal_plan, optimal_basis = simplex_main_phase(A_initial,
c_initial, x_initial, B_initial)

    if optimal_plan is not None:
        print("--- Результат ---")
        print(f"Оптимальный план: {np.round(optimal_plan, 4)}")
        print(f"Оптимальный базис: {[i+1 for i in optimal_basis]}")
        z_value = c_initial @ optimal_plan
        print(f"Максимальное значение целевой функции: {z_value:.4f}")
```